

39. GPU 候補の厳密検証

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	5
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	6
6.2 名前、代入、型、参照	6
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	7
6.5 例外、try/finally、with、資源解放	7
6.6 list、dict、deque、NumPy 配列、pandas DataFrame	7
6.7 CLI、PowerShell、Bash、環境変数、終了コード	7
6.8 freshness、filter、guard、fallback、fail-safe	8
6.9 CSV/YAML の data contract と検証	8
6.10 単体試験、SILS、replay、観測可能性	8
6.11 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明	8
7 関数・クラスを上から順に解説	9
7.1 L26 関数 resolve_result_path	9
7.2 L31 関数 resolve_profile_asset	9

7.3	L36 関数 <code>evaluate_event_timing</code>	10
7.4	L112 関数 <code>exact_replay_signature</code>	13
7.5	L137 関数 <code>inspect_prediction_mesh</code>	14
7.6	L199 関数 <code>prediction_execution_soc_errors</code>	16
7.7	L228 関数 <code>evaluate_soc_guard_intervention</code>	18
7.8	L249 関数 <code>main</code>	19
7.9	CLI 引数	23
8	処理の流れ	23
9	このファイルの位置づけ	23

1 対象

項目	内容
ファイル	scripts/validate_gpu_upper_policy_candidates.py
ソース SHA-256	5bb6f5b03980f4ccf6c741e6c2dbe7b09c7618c176f45c96bb6e471a7b33d4e6
種別	Python
区分	planning validation
役割	GPU 探索で得た speed policy 候補を CPU 側 exact replay と gate で再判定する。
起動文脈	GPU acceptance の中心。

2 このファイルを読む理由

GPU 探索で得た speed policy 候補を CPU 側 exact replay と gate で再判定する。

- numerical match、mission feasibility、gate pass を確認する。

3 呼び出し関係

3.1 どこから呼ばれるか

- GPU acceptance pipeline
- 手動検証

3.2 次に読むと理解が進むファイル

- scripts/run_upper_mesh_convergence.py
- scripts/solar_sim.py

3.3 同一リポジトリ内の主要依存

- scripts/run_upper_mesh_convergence.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

Replay every GPU policy in the exact 1 Hz simulator and select the fastest feasible one.

4.1 代表コード断片

```
ROOT = Path(__file__).resolve().parents[1]

def resolve_result_path(raw: str) -> Path:
    path = Path(str(raw))
    return path if path.is_absolute() else (ROOT / path).resolve()

def resolve_profile_asset(profile_path: Path, raw: str) -> Path:
```

```

path = Path(str(raw))
return path if path.is_absolute() else (profile_path.parent / path).resolve()

```

```

def evaluate_event_timing(detail: pd.DataFrame, cfg: dict, profile_path: Path) -> dict:
    """Check official control-stop closing times and the absolute finish deadline."""
    paths = cfg.get("paths", {}) or {}
    simulation = cfg.get("simulation", {}) or {}
    stop_path = resolve_profile_asset(profile_path, paths.get("stop_yaml", ""))
    stop_cfg = (
        yaml.safe_load(stop_path.read_text(encoding="utf-8-sig")) or {}
        if stop_path.is_file()
        else {}
    )

```

5 主要構造の読み取り

主要関数は `resolve_result_path`, `resolve_profile_asset`, `evaluate_event_timing`, `exact_replay_signature`, `inspect_prediction_mesh`, `prediction_execution_soc_errors`, `evaluate_soc_guard_intervention`, `main`。CLI 引数宣言は 8 件。

5.1 主要クラス・関数

行	種別	名前	補足
26	function	<code>resolve_result_path</code>	args: raw
31	function	<code>resolve_profile_asset</code>	args: profile_path, raw
36	function	<code>evaluate_event_timing</code>	args: detail, cfg, profile_path
112	function	<code>exact_replay_signature</code>	args: profile, policy
137	function	<code>inspect_prediction_mesh</code>	args: manifest, manifest_path
199	function	<code>prediction_execution_soc_errors</code>	args: manifest
228	function	<code>evaluate_soc_guard_intervention</code>	args: detail, manifest
249	function	<code>main</code>	

5.2 ファイルを上から読んだときの定義順

行	内容
17	例外処理を伴う <code>try</code> ブロックを実行する。
23	<code>ROOT</code> に <code>Path(__file__).resolve().parents[1]</code> の結果を代入する。
26	関数 <code>resolve_result_path</code> を定義する。
31	関数 <code>resolve_profile_asset</code> を定義する。
36	関数 <code>evaluate_event_timing</code> を定義する。
112	関数 <code>exact_replay_signature</code> を定義する。
137	関数 <code>inspect_prediction_mesh</code> を定義する。
199	関数 <code>prediction_execution_soc_errors</code> を定義する。
228	関数 <code>evaluate_soc_guard_intervention</code> を定義する。

249 関数 `main` を定義する。
476 条件 `__name__ == '__main__'` を判定し、真なら内部処理を行う。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
4	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
6	<code>import argparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L250。
7	<code>import json</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L309, L340, L470, L472。
8	<code>import math</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L148, L205, L210。
9	<code>import shutil</code>	shutil モジュールを利用するため。このファイル内での主な使用位置は L458。
10	<code>import subprocess</code>	git 情報取得や外部コマンド実行を行うため。このファイル内での主な使用位置は L318, L322。
11	<code>import sys</code>	sys モジュールを利用するため。このファイル内での主な使用位置は L319。
12	<code>from pathlib import Path</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L23, L26, L27, L31, L32, L36, L112, L123, ...。
14	<code>import pandas as pd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L36, L48, L49, L50, L63, L64, L88, L89, ...。
15	<code>import yaml</code>	profile、stop、schedule、summary YAML を読み書きするため。このファイル内での主な使用位置は L42, L127, L272, L297。
18	<code>from scripts.run_upper_ mesh_convergence import file_ sha256, materialize_profile</code>	upper mesh 収束確認から file_sha256, materialize_profile を読み込み、このファイルの内部処理を分担させるため。実体ファイルは scripts/run_upper_mesh_convergence.py。このファイル内での主な使用位置は L125, L126, L133, L282。
20	<code>from run_upper_mesh_ convergence import file_ sha256, materialize_profile</code>	run_upper_mesh_convergence から file_sha256, materialize_profile を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L125, L126, L133, L282。
113	<code>import hashlib</code>	snapshot ID や入力資産の digest を作るため。このファイル内での主な使用位置は L134。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.7 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。‘argparse’は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS_DOMAIN_ID、RMW_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

6.8 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.9 CSV/YAML の data contract と検証

data contract は列名、型、単位、timezone、欠損可否、並び順、重複、許容範囲、先頭行、encoding を事前に決めた仕様である。単に CSV として読めることは、モデル入力として正しいことを意味しない。

同定用実測、map、route、forecast、stop、schedule はそれぞれ grain が異なる。生成時に schema validation、物理範囲、時間単調性、route 範囲、coverage を検査し、検査結果を artifact として残す。

学習用データと独立検証データを分離し、RMSE だけでなく bias、時系列残差、energy 積算誤差、終端 SoC、温度・電圧制約、外挿領域を評価する。

6.10 単体試験、SILS、replay、観測可能性

単体試験は関数やモデル項の局所契約、SILS は複数 Node と時間進行を含む閉ループ、historical replay は実測入力への再現性、本番 preflight は実機通信と停止動作を確認する。目的が異なるため一つで代用しない。

再現可能な診断には、入力、出力、内部状態、solver 成否、候補数、計算時間、fallback、source revision、profile hash を同じ run ID で記録する。

非同期不具合は最終値だけでは追えない。topic 周期、message age、callback 所要時間、queue 遅延、Node 生存、publisher 数を同時に観測する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 公式: Executors](#)

6.11 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は cost 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 seed や大域探索と組み合わせる。success が False でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた sampling と局所最適化を組み合わせる大域最適化法である。有限 Cartesian grid の全列挙は、その grid 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)
- SciPy 公式: [scipy.optimize.shgo](#)

7 関数・クラスを上から順に解説

7.1 L26 関数 `resolve_result_path`

- 行範囲: L26–L28
- 所属: module 直下
- 定義: `resolve_result_path(raw:str)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `resolve`, `str`
- 戻り値の要点: `path if path.is_absolute() else (ROOT / path).resolve()`
- 主なローカル代入名: `path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `path` に `Path(str(raw))` の結果を代入する。
2. `path if path.is_absolute() else (ROOT / path).resolve()` を返す。

代表コード断片

```
def resolve_result_path(raw: str) -> Path:
    path = Path(str(raw))
    return path if path.is_absolute() else (ROOT / path).resolve()
```

7.2 L31 関数 `resolve_profile_asset`

- 行範囲: L31–L33
- 所属: module 直下
- 定義: `resolve_profile_asset(profile_path:Path,raw:str)->Path`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `resolve`, `str`

- 戻り値の要点: `path if path.is_absolute() else (profile_path.parent / path).resolve()`
- 主なローカル代入名: `path`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `path` に `Path(str(raw))` の結果を代入する。
2. `path if path.is_absolute() else (profile_path.parent / path).resolve()` を返す。

代表コード断片

```
def resolve_profile_asset(profile_path: Path, raw: str) -> Path:
    path = Path(str(raw))
    return path if path.is_absolute() else (profile_path.parent / path).resolve()
```

7.3 L36 関数 `evaluate_event_timing`

- 行範囲: L36–L109
- 所属: `module` 直下
- 定義: `evaluate_event_timing(detail:pd.DataFrame, cfg:dict, profile_path:Path)->dict`
- docstring: Check official control-stop closing times and the absolute finish deadline.
- このブロックが直接呼ぶ主な関数・メソッド: `DataFrame`, `Timestamp`, `all`, `append`, `bool`, `dropna`, `float`, `get`, `is_file`, `isna`, `isoformat`, `max`
- 戻り値の要点: `{'control_stop_windows_pass': bool(all_stops_observed and all((row['passed'] for row in stop_results))), 'finish_deadline_pass': bool(deadline_late_sec <= 1.0), 'max_control_stop_late_sec': max_late_sec, 'finish_deadline_late_sec': deadline_late_sec, 'finish_time_utc': finish_time.isoformat() if not pd.isna(finish_time) else "", 'race_deadline_utc': deadline.isoformat() if deadline_raw else "", 'control_stops': stop_results}`
- 主なローカル代入名: `all_stops_observed`, `arrival`, `arrivals`, `close`, `close_raw`, `deadline`, `deadline_late_sec`, `deadline_raw`, `distance`, `distance_col`, `finish_cfg`, `finish_time`, `late_sec`, `max_late_sec`, `observed`, `ordered`, `paths`, `row`, `simulation`, `stop`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `paths` に `cfg.get('paths', {})` or `{}` の結果を代入する。
2. `simulation` に `cfg.get('simulation', {})` or `{}` の結果を代入する。
3. `stop_path` に `resolve_profile_asset(profile_path, paths.get('stop_yaml', ''))` の結果を代入する。
4. `stop_cfg` に `yaml.safe_load(stop_path.read_text(encoding='utf-8-sig'))` or `{}` if `stop_path.is_file()` else `{}` の結果を代入する。
5. `distance_col` に `'s_end_km'` if `'s_end_km'` in `detail` else `'s_km'` の結果を代入する。
6. `time_col` に `'time_end_utc'` if `'time_end_utc'` in `detail` else `'time_utc'` の結果を代入する。
7. `distance` に `pd.to_numeric(detail.get(distance_col), errors='coerce')` の結果を代入する。
8. `times` に `pd.to_datetime(detail.get(time_col), errors='coerce', utc=True, format='mixed')` の結果を代入する。
9. `ordered` に `pd.DataFrame({'s_km': distance, 'time': times}).dropna().sort_values('time')` の結果を代入する。
10. `stop_results` に `[]` の結果を代入する。
11. `max_late_sec` に `0.0` の結果を代入する。
12. `all_stops_observed` に `True` の結果を代入する。
13. `stop_cfg.get('stops', [])` or `[]` を順に走査し、各要素を `stop` に入れて処理する。
14. `close_raw` に `str(stop.get('window_close_utc', ''))` or `''` の結果を代入する。
15. 条件 `not close_raw` を判定し、真なら内部処理を行う。
16. `Continue` 文を実行する。
17. `stop_km` に `float(stop.get('s_km', 0.0))` の結果を代入する。
18. `arrivals` に `ordered.loc[ordered['s_km'] >= stop_km - 1e-06, 'time']` の結果を代入する。
19. `observed` に `not arrivals.empty` の結果を代入する。
20. `all_stops_observed` に `all_stops_observed and observed` の結果を代入する。
21. `arrival` に `arrivals.iloc[0]` if `observed` else `pd.NaT` の結果を代入する。
22. `close` に `pd.Timestamp(close_raw)` の結果を代入する。
23. `close` に `close.tz_localize('UTC')` if `close.tzinfo is None` else `close.tz_convert('UTC')` の結果を代入する。
24. `late_sec` に `max(0.0, float((arrival - close).total_seconds()))` if `observed` else `float('inf')` の結果を代入する。
25. `max_late_sec` に `max(max_late_sec, late_sec)` の結果を代入する。
26. `stop_results.append(...)` を実行する。
27. `finish_cfg` に `stop_cfg.get('finish', {})` or `{}` の結果を代入する。
28. `deadline_raw` に `str(simulation.get('race_deadline_utc', finish_cfg.get('window_close_utc', '')))` or `''` の結果を代入する。

29. `finish_time` に `ordered['time'].iloc[-1]` if not `ordered.empty` else `pd.NaT` の結果を代入する。
30. `deadline` に `pd.Timestamp(deadline_raw)` if `deadline_raw` else `pd.NaT` の結果を代入する。
31. 条件 `deadline_raw` and `deadline.tzinfo` is `None` を判定し、真なら内部処理を行う。
32. `deadline` に `deadline.tz_localize('UTC')` の結果を代入する。
33. 上の条件が偽の場合:
34. 条件 `deadline_raw` を判定し、真なら内部処理を行う。
35. `deadline` に `deadline.tz_convert('UTC')` の結果を代入する。
36. `deadline_late_sec` に `max(0.0, float((finish_time - deadline).total_seconds()))` if `deadline_raw` and (not `pd.isna(finish_time)`) else `0.0` if not `deadline_raw` else `float('inf')` の結果を代入する。
37. `{'control_stop_windows_pass': bool(all_stops_observed and all((row['passed'] for row in stop_results))), 'finish_deadline_pass': bool(deadline_late_sec <= 1.0), 'max_control_stop_late_sec': max_late_sec, 'finish_deadline_late_sec': deadline_late_sec, 'finish_time_utc': finish_time.isoformat() if not pd.isna(finish_time) else "", 'race_deadline_utc': deadline.isoformat() if deadline_raw else "", 'control_stops': stop_results}` を返す。

代表コード断片

```
def evaluate_event_timing(detail: pd.DataFrame, cfg: dict, profile_path: Path) -> dict:
    """Check official control-stop closing times and the absolute finish deadline."""
    paths = cfg.get("paths", {}) or {}
    simulation = cfg.get("simulation", {}) or {}
    stop_path = resolve_profile_asset(profile_path, paths.get("stop_yaml", ""))
    stop_cfg = (
        yaml.safe_load(stop_path.read_text(encoding="utf-8-sig")) or {}
        if stop_path.is_file()
        else {}
    )
    distance_col = "s_end_km" if "s_end_km" in detail else "s_km"
    time_col = "time_end_utc" if "time_end_utc" in detail else "time_utc"
    distance = pd.to_numeric(detail.get(distance_col), errors="coerce")
    times = pd.to_datetime(detail.get(time_col), errors="coerce", utc=True, format="mixed")
    ordered = pd.DataFrame({"s_km": distance, "time": times}).dropna().sort_values("time")

    stop_results = []
    max_late_sec = 0.0
    all_stops_observed = True
    for stop in stop_cfg.get("stops", []) or []:
        close_raw = str(stop.get("window_close_utc", "")) or ""
        if not close_raw:
            continue
        stop_km = float(stop.get("s_km", 0.0))
        arrivals = ordered.loc[ordered["s_km"] >= stop_km - 1.0e-6, "time"]
        observed = not arrivals.empty
        all_stops_observed = all_stops_observed and observed
        arrival = arrivals.iloc[0] if observed else pd.NaT
        close = pd.Timestamp(close_raw)
        close = close.tz_localize("UTC") if close.tzinfo is None else close.tz_convert("UTC")
    )
    late_sec = (
        max(0.0, float((arrival - close).total_seconds()))
        if observed
        else float("inf")
    )
    ...
```

7.4 L112 関数 `exact_replay_signature`

- 行範囲: L112–L134
- 所属: module 直下
- 定義: `exact_replay_signature(profile:Path,policy:Path)->str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `encode`, `file_sha256`, `get`, `hexdigest`, `is_file`, `isinstance`, `join`, `read_text`, `resolve`, `resolve_profile_asset`, `safe_load`
- 戻り値の要点: `hashlib.sha256(payload.encode('ascii')).hexdigest()`
- 主なローカル代入名: `asset`, `cfg`, `dependencies`, `path`, `payload`, `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. Import 文を実行する。
2. `dependencies` に `(ROOT / 'scripts' / 'solar_sim.py', ROOT / 'mpc_solarcar' / 'upper_horizon.py', ROOT / 'mpc_solarcar' / 'upper_solver.py', ROOT / 'mpc_solarcar' / 'upper_policy.py', ROOT / 'mpc_solarcar' / 'upper_cost.py', ROOT / 'mpc_solarcar' / 'model.py', ROOT / 'mpc_solarcar' / 'signal_utils.py', Path(__file__).resolve())` の結果を代入する。
3. `payload` に `file_sha256(profile) + file_sha256(policy)` の結果を代入する。
4. `payload` を `Add` で更新する。
5. `cfg` に `yaml.safe_load(profile.read_text(encoding='utf-8-sig')) or {}` の結果を代入する。
6. `(cfg.get('paths', {}) or {}).values()` を順に走査し、各要素を `raw` に入れて処理する。
7. 条件 `not isinstance(raw, str) or not raw.strip()` を判定し、真なら内部処理を行う。
8. `Continue` 文を実行する。
9. `asset` に `resolve_profile_asset(profile, raw)` の結果を代入する。
10. 条件 `asset.is_file()` を判定し、真なら内部処理を行う。
11. `payload` を `Add` で更新する。
12. `hashlib.sha256(payload.encode('ascii')).hexdigest()` を返す。

代表コード断片

```
def exact_replay_signature(profile: Path, policy: Path) -> str:
    import hashlib

    dependencies = (
        ROOT / "scripts" / "solar_sim.py",
        ROOT / "mpc_solarcar" / "upper_horizon.py",
        ROOT / "mpc_solarcar" / "upper_solver.py",
        ROOT / "mpc_solarcar" / "upper_policy.py",
        ROOT / "mpc_solarcar" / "upper_cost.py",
        ROOT / "mpc_solarcar" / "model.py",
        ROOT / "mpc_solarcar" / "signal_utils.py",
        Path(__file__).resolve(),
    )
    payload = file_sha256(profile) + file_sha256(policy)
    payload += "".join(file_sha256(path) for path in dependencies)
    cfg = yaml.safe_load(profile.read_text(encoding="utf-8-sig")) or {}
    for raw in (cfg.get("paths", {}) or {}).values():
        if not isinstance(raw, str) or not raw.strip():
            continue
        asset = resolve_profile_asset(profile, raw)
        if asset.is_file():
            payload += file_sha256(asset)
    return hashlib.sha256(payload.encode("ascii")).hexdigest()
```

7.5 L137 関数 inspect_prediction_mesh

- 行範囲: L137–L196
- 所属: module 直下
- 定義: inspect_prediction_mesh(manifest:dict,manifest_path:Path,*,requested_ds_km:float,start_s_km:float,race_km:float)->dict
- docstring: Verify that the selected prediction trace used the requested distance mesh.
- このブロックが直接呼ぶ主な関数・メソッド: Path, abs, bool, ceil, dropna, float, get, int, is_file, len, list, max
- 戻り値の要点: result / result / result / result
- 主なローカル代入名: diagnostic, diagnostics, ends, expected_min_steps, max_trace_ds_km, positive, raw_trace, remaining_km, requested_ds_km, result, segment_km, starts, terminal_km, tolerance_km, trace, trace_path
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 5、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `requested_ds_km` に `float(requested_ds_km)` の結果を代入する。
2. `remaining_km` に `max(0.0, float(race_km) - float(start_s_km))` の結果を代入する。
3. `expected_min_steps` に `int(math.ceil(remaining_km / requested_ds_km - 1e-09))` の結果を代入する。
4. `result` に `{'valid': False, 'expected_min_steps': expected_min_steps, 'prediction_steps': 0, 'trace_drive_steps': 0, 'max_trace_ds_km': float('nan'), 'trace_terminal_km': float('nan')}` の結果を代入する。
5. `diagnostics` に `list(manifest.get('upper_solver_diagnostics', []) or [])` の結果を代入する。
6. 条件 `not diagnostics` を判定し、真なら内部処理を行う。
7. `result` を返す。
8. `diagnostic` に `diagnostics[0]` の結果を代入する。
9. `result['prediction_steps']` に `int(diagnostic.get('prediction_steps', 0) or 0)` の結果を代入する。
10. `raw_trace` に `str(diagnostic.get('selected_prediction_trace_csv', '') or '')` の結果を代入する。
11. 条件 `not raw_trace` を判定し、真なら内部処理を行う。
12. `result` を返す。
13. `trace_path` に `resolve_result_path(raw_trace)` の結果を代入する。
14. 条件 `not trace_path.is_file()` を判定し、真なら内部処理を行う。
15. `trace_path` に `manifest_path.parent / Path(raw_trace).name` の結果を代入する。
16. 条件 `not trace_path.is_file()` を判定し、真なら内部処理を行う。
17. `result` を返す。
18. `trace` に `pd.read_csv(trace_path, usecols=['s_km', 's_end_km'])` の結果を代入する。
19. `starts` に `pd.to_numeric(trace['s_km'], errors='coerce')` の結果を代入する。
20. `ends` に `pd.to_numeric(trace['s_end_km'], errors='coerce')` の結果を代入する。
21. `segment_km` に `(ends - starts).abs()` の結果を代入する。
22. `positive` に `segment_km[segment_km > 1e-09]` の結果を代入する。
23. 条件 `positive.empty` を判定し、真なら内部処理を行う。
24. `result` を返す。
25. `max_trace_ds_km` に `float(positive.max())` の結果を代入する。
26. `terminal_km` に `float(ends.dropna().iloc[-1])` の結果を代入する。
27. `result.update(...)` を実行する。
28. `tolerance_km` に `max(1e-08, requested_ds_km * 1e-06)` の結果を代入する。
29. `result['valid']` に `bool(result['prediction_steps'] >= expected_min_steps and result['trace_drive_steps'] >= expected_min_steps and (max_trace_ds_km <= requested_ds_km + tolerance_km) and (abs(terminal_km - float(race_km)) <= tolerance_km))` の結果を代入する。
30. `result` を返す。

代表コード断片

```
def inspect_prediction_mesh(
    manifest: dict,
    manifest_path: Path,
    *,
    requested_ds_km: float,
    start_s_km: float,
    race_km: float,
) -> dict:
    """Verify that the selected prediction trace used the requested distance mesh."""
    requested_ds_km = float(requested_ds_km)
    remaining_km = max(0.0, float(race_km) - float(start_s_km))
    expected_min_steps = int(math.ceil(remaining_km / requested_ds_km - 1.0e-9))
    result = {
        "valid": False,
        "expected_min_steps": expected_min_steps,
        "prediction_steps": 0,
        "trace_drive_steps": 0,
        "max_trace_ds_km": float("nan"),
        "trace_terminal_km": float("nan"),
    }
    diagnostics = list(manifest.get("upper_solver_diagnostics", []) or [])
    if not diagnostics:
        return result
    diagnostic = diagnostics[0]
    result["prediction_steps"] = int(diagnostic.get("prediction_steps", 0) or 0)

    raw_trace = str(diagnostic.get("selected_prediction_trace_csv", "") or "")
    if not raw_trace:
        return result
    trace_path = resolve_result_path(raw_trace)
    if not trace_path.is_file():
        trace_path = manifest_path.parent / Path(raw_trace).name
    if not trace_path.is_file():
        return result

    ...
```

7.6 L199 関数 prediction_execution_soc_errors

- 行範囲: L199–L225
- 所属: module 直下
- 定義: prediction_execution_soc_errors(manifest:dict)->dict
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: append, dict, float, get, int, isfinite, list
- 戻り値の要点: {'initial_prediction_soc': initial_prediction, 'latest_nontrivial_prediction_soc': latest_prediction, 'initial_error': final_soc - initial_prediction, 'latest_nontrivial_error': final_soc - latest_prediction} / {'initial_prediction_soc': float('nan'), 'latest_nontrivial_prediction_soc': float('nan'), 'initial_error': float('inf'), 'latest_nontrivial_error': float('inf')}
- 主なローカル代入名: diagnostic, diagnostics, final_soc, initial_prediction, latest_prediction, nontrivial, predictions, selected, soc, steps, terminal_soc
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。

- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. `diagnostics` に `list(manifest.get('upper_solver_diagnostics', []) or [])` の結果を代入する。
2. `predictions` に `[]` の結果を代入する。
3. `diagnostics` を順に走査し、各要素を `diagnostic` に入れて処理する。
4. `selected` に `dict(diagnostic.get('selected_prediction', {}) or {})` の結果を代入する。
5. `terminal_soc` に `float(selected.get('terminal_soc', float('nan')))` の結果を代入する。
6. 条件 `math.isfinite(terminal_soc)` を判定し、真なら内部処理を行う。
7. `predictions.append(...)` を実行する。
8. `final_soc` に `float(manifest.get('final_soc', float('nan')))` の結果を代入する。
9. 条件 `not predictions or not math.isfinite(final_soc)` を判定し、真なら内部処理を行う。
10. `{'initial_prediction_soc': float('nan'), 'latest_nontrivial_prediction_soc': float('nan'), 'initial_error': float('inf'), 'latest_nontrivial_error': float('inf')}` を返す。
11. `initial_prediction` に `predictions[0][1]` の結果を代入する。
12. `nontrivial` に `[soc for steps, soc in predictions if steps > 1]` の結果を代入する。
13. `latest_prediction` に `nontrivial[-1] if nontrivial else predictions[-1][1]` の結果を代入する。
14. `{'initial_prediction_soc': initial_prediction, 'latest_nontrivial_prediction_soc': latest_prediction, 'initial_error': final_soc - initial_prediction, 'latest_nontrivial_error': final_soc - latest_prediction}` を返す。

代表コード断片

```
def prediction_execution_soc_errors(manifest: dict) -> dict:
    diagnostics = list(manifest.get("upper_solver_diagnostics", []) or [])
    predictions = []
    for diagnostic in diagnostics:
        selected = dict(diagnostic.get("selected_prediction", {}) or {})
        terminal_soc = float(selected.get("terminal_soc", float("nan")))
        if math.isfinite(terminal_soc):
            predictions.append(
                (int(diagnostic.get("prediction_steps", 0) or 0), terminal_soc)
            )
    final_soc = float(manifest.get("final_soc", float("nan")))
    if not predictions or not math.isfinite(final_soc):
        return {
            "initial_prediction_soc": float("nan"),
            "latest_nontrivial_prediction_soc": float("nan"),
            "initial_error": float("inf"),
            "latest_nontrivial_error": float("inf"),
        }
```

```

initial_prediction = predictions[0][1]
nontrivial = [soc for steps, soc in predictions if steps > 1]
latest_prediction = nontrivial[-1] if nontrivial else predictions[-1][1]
return {
    "initial_prediction_soc": initial_prediction,
    "latest_nontrivial_prediction_soc": latest_prediction,
    "initial_error": final_soc - initial_prediction,
    "latest_nontrivial_error": final_soc - latest_prediction,
}

```

7.7 L228 関数 `evaluate_soc_guard_intervention`

- 行範囲: L228–L246
- 所属: module 直下
- 定義: `evaluate_soc_guard_intervention(detail:pd.DataFrame,manifest:dict)->dict`
- docstring: Reject policies that only finish because the execution safety guard intervened.
- このブロックが直接呼ぶ主な関数・メソッド: `astype`, `bool`, `fillna`, `float`, `get`, `int`, `isin`, `lower`, `strip`, `sum`, `to_numeric`
- 戻り値の要点: `{'intervention_rows': intervention_rows, 'intervention_sec': intervention_sec, 'passed': bool(intervention_rows == 0 and intervention_sec <= 1e-09)}`
- 主なローカル代入名: `active`, `dt_sec`, `intervention_rows`, `intervention_sec`, `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件‘`soc_guard_intervened`’ in `detail.columns` を判定し、真なら内部処理を行う。
2. `raw` に `detail['soc_guard_intervened']` の結果を代入する。
3. `active` に `raw.astype(str).str.strip().str.lower().isin({'true', '1', 'yes'})` の結果を代入する。
4. 条件‘`step_dt_sec`’ in `detail.columns` を判定し、真なら内部処理を行う。
5. `dt_sec` に `pd.to_numeric(detail['step_dt_sec'], errors='coerce').fillna(0.0)` の結果を代入する。
6. `intervention_sec` に `float(dt_sec.loc[active].sum())` の結果を代入する。
7. 上の条件が偽の場合:
8. `intervention_sec` に `float(active.sum())` の結果を代入する。
9. `intervention_rows` に `int(active.sum())` の結果を代入する。
10. 上の条件が偽の場合:

11. `intervention_rows` に `int(manifest.get('soc_guard_intervention_rows', 0) or 0)` の結果を代入する。
12. `intervention_sec` に `float(manifest.get('soc_guard_intervention_sec', 0.0) or 0.0)` の結果を代入する。
13. `{'intervention_rows': intervention_rows, 'intervention_sec': intervention_sec, 'passed': bool(intervention_rows == 0 and intervention_sec <= 1e-09)}` を返す。

代表コード断片

```
def evaluate_soc_guard_intervention(detail: pd.DataFrame, manifest: dict) -> dict:
    """Reject policies that only finish because the execution safety guard intervened."""
    if "soc_guard_intervened" in detail.columns:
        raw = detail["soc_guard_intervened"]
        active = raw.astype(str).str.strip().str.lower().isin({"true", "1", "yes"})
        if "step_dt_sec" in detail.columns:
            dt_sec = pd.to_numeric(detail["step_dt_sec"], errors="coerce").fillna(0.0)
            intervention_sec = float(dt_sec.loc[active].sum())
        else:
            intervention_sec = float(active.sum())
        intervention_rows = int(active.sum())
    else:
        intervention_rows = int(manifest.get("soc_guard_intervention_rows", 0) or 0)
        intervention_sec = float(manifest.get("soc_guard_intervention_sec", 0.0) or 0.0)
    return {
        "intervention_rows": intervention_rows,
        "intervention_sec": intervention_sec,
        "passed": bool(intervention_rows == 0 and intervention_sec <= 1.0e-9),
    }
```

7.8 L249 関数 main

- 行範囲: L249–L473
- 所属: module 直下
- 定義: `main()->int`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ArgumentParser`, `DataFrame`, `FileNotFoundError`, `Path`, `abs`, `add_argument`, `all`, `append`, `bool`, `copy2`, `dumps`, `evaluate_event_timing`
- 戻り値の要点: 0 if `selection['selected']` else 2
- 主なローカル代入名: `args`, `base_cfg`, `base_profile`, `cached`, `cached_detail`, `campaign`, `cfg`, `checks`, `column`, `detail`, `detail_columns`, `detail_path`, `event_timing`, `feasible`, `final_soc`, `label`, `log`, `manifest`, `manifest_path`, `max_charge_a`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: `FileNotFoundError(f'No {args.stage} policies under {campaign}')`
- 制御構造の規模: 条件分岐 5、ループ 1、try 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。
- `with` 文は `context manager` に開始・終了処理を任せ、ファイルなどの資源を確実に解放する。

上から順の処理

1. parser に `argparse.ArgumentParser(description=__doc__)` の結果を代入する。
2. `parser.add_argument(...)` を実行する。
3. `parser.add_argument(...)` を実行する。
4. `parser.add_argument(...)` を実行する。
5. `parser.add_argument(...)` を実行する。
6. `parser.add_argument(...)` を実行する。
7. `parser.add_argument(...)` を実行する。
8. `parser.add_argument(...)` を実行する。
9. `parser.add_argument(...)` を実行する。
10. args に `parser.parse_args()` の結果を代入する。
11. `base_profile` に `args.profile.resolve()` の結果を代入する。
12. `campaign` に `args.campaign_dir.resolve()` の結果を代入する。
13. `output` に `args.output_dir.resolve()` の結果を代入する。
14. `seed_pattern` に `args.seed_label.strip()` or `'seed_*` の結果を代入する。
15. `policies` に `sorted(campaign.glob(f'{seed_pattern}/{args.stage}/latest_policy.csv'))` の結果を代入する。
16. 条件 `not policies` を判定し、真なら内部処理を行う。
17. `FileNotFoundError(f'No {args.stage} policies under {campaign}')` を送出する。
18. `base_cfg` に `yaml.safe_load(base_profile.read_text(encoding='utf-8'))` or `{}` の結果を代入する。
19. `model` に `base_cfg['model']` の結果を代入する。
20. `rows` に `[]` の結果を代入する。
21. `policies` を順に走査し、各要素を `policy` に入れて処理する。
22. `label` に `policy.parents[1].name` の結果を代入する。
23. `run_dir` に `output / label` の結果を代入する。
24. `profile_path` に `run_dir / 'profile_exact_1hz.yaml'` の結果を代入する。
25. `manifest_path` に `run_dir / 'latest_simulation_run.json'` の結果を代入する。
26. `signature_path` に `run_dir / 'run_signature.txt'` の結果を代入する。
27. `cfg` に `materialize_profile(base_profile, profile_path, policy_path=policy, ds_km=float(args.integration_ds_km), ctrl_km=float(args.control_ds_km))` の結果を代入する。
28. `cfg['simulation']['require_model_validation_gate']` に `False` の結果を代入する。
29. `cfg['simulation']['validation_scope']` に `'research_only_unvalidated_model'` の結果を代入する。
30. `notes` に `list(cfg.setdefault('meta', {}), get('notes', []) or [])` の結果を代入する。
31. `notes.append(...)` を実行する。
32. `cfg['meta']['notes']` に `list(dict.fromkeys(notes))` の結果を代入する。

33. `profile_path.write_text(...)` を実行する。
34. `signature` に `exact_replay_signature(profile_path, policy)` の結果を代入する。
35. `reusable` に `bool(not args.no_resume and manifest_path.is_file() and signature_path.is_file() and (signature_path.read_text(encoding='ascii').strip() == signature))` の結果を代入する。
36. 条件 `reusable` を判定し、真なら内部処理を行う。
37. `cached` に `json.loads(manifest_path.read_text(encoding='utf-8'))` の結果を代入する。
38. `cached_detail` に `resolve_result_path(str(cached.get('detail_csv', '')))` の結果を代入する。
39. `reusable` に `bool(cached_detail.is_file() and int(cached.get('detail_rows', 0) or 0) > 0)` の結果を代入する。
40. 条件 `not reusable` を判定し、真なら内部処理を行う。
41. `run_dir.mkdir(...)` を実行する。
42. `with` 文で `(run_dir / 'console.log').open('w', encoding='utf-8')` を管理しながら処理する。
43. `result` に `subprocess.run([sys.executable, '-u', str(ROOT / 'scripts' / 'solar_sim.py'), '-profile_yaml', str(profile_path)], cwd=ROOT, stdout=log, stderr=subprocess.STDOUT, check=False, text=True)` の結果を代入する。
44. 条件 `result.returncode != 0 or not manifest_path.is_file()` を判定し、真なら内部処理を行う。
45. `rows.append(...)` を実行する。
46. `Continue` 文を実行する。
47. `signature_path.write_text(...)` を実行する。
48. `manifest` に `json.loads(manifest_path.read_text(encoding='utf-8'))` の結果を代入する。
49. `mesh` に `inspect_prediction_mesh(manifest, manifest_path, requested_ds_km=float(args.integration_ds_km), start_s_km=float(cfg.get('simulation', {}).get('start_s_km', 0.0)), race_km=float(cfg.get('mpc', {}).get('race_km', manifest.get('race_km', 0.0))))` の結果を代入する。
50. `detail_path` に `resolve_result_path(manifest['detail_csv'])` の結果を代入する。
51. `detail_columns` に `set(pd.read_csv(detail_path, nrows=0).columns)` の結果を代入する。
52. `detail` に `pd.read_csv(detail_path, usecols=[column for column in ('I', 'V', 's_km', 's_end_km', 'time_utc', 'time_end_utc', 'step_dt_sec', 'soc_guard_intervened') if column in detail_columns])` の結果を代入する。
53. `max_discharge_a` に `float(pd.to_numeric(detail['I'], errors='coerce').max())` の結果を代入する。
54. `max_charge_a` に `float(pd.to_numeric(detail['I'], errors='coerce').min())` の結果を代入する。
55. `min_voltage_v` に `float(pd.to_numeric(detail['V'], errors='coerce').min())` の結果を代入する。
56. `max_voltage_v` に `float(pd.to_numeric(detail['V'], errors='coerce').max())` の結果を代入する。
57. `soc_floor` に `float(cfg['mpc'].get('terminal_soc_min', model.get('soc_min', 0.1)))` の結果を代入する。
58. `soc_target` に `float(cfg['mpc'].get('soc_finish_target', soc_floor))` の結果を代入する。
59. `soc_target_tolerance` に `float(cfg['mpc'].get('soc_finish_tol', 0.005))` の結果を代入する。
60. `sync_tol` に `float(cfg['mpc'].get('prediction_execution_soc_tolerance', 0.002))` の結果を代入する。
61. `final_soc` に `float(manifest.get('final_soc', float('nan')))` の結果を代入する。
62. `soc_sync` に `prediction_execution_soc_errors(manifest)` の結果を代入する。

63. `sync_error` に `abs(float(soc_sync['initial_error']))` の結果を代入する。
64. `event_timing` に `evaluate_event_timing(detail, cfg, profile_path)` の結果を代入する。
65. `soc_guard` に `evaluate_soc_guard_intervention(detail, manifest)` の結果を代入する。
66. `checks` に `{'finish': bool(manifest.get('finish_reached', False)), 'soc': float(manifest.get('min_soc', -1.0)) >= soc_floor - 0.0001, 'terminal_target': soc_target - soc_target_tolerance - 0.0001 <= final_soc <= soc_target + soc_target_tolerance + 0.0001, 'discharge_current': max_discharge_a <= float(model.get('I_max', 40.0)) + 0.1, 'charge_current': max_charge_a >= float(model.get('I_chg_min', -16.5)) - 0.1, 'voltage_min': min_voltage_v >= float(model.get('V_min', 0.0)) - 0.1, 'voltage_max': max_voltage_v <= float(model.get('V_max', float('inf')) + 0.1, 'prediction_execution_sync': sync_error <= sync_tol, 'prediction_mesh': bool(mesh['valid']), 'control_stop_windows': bool(event_timing['control_stop_windows_pass']), 'finish_deadline': bool(event_timing['finish_deadline_pass']), 'no_soc_guard_intervention': bool(soc_guard['passed'])}` の結果を代入する。
67. `rows.append(...)` を実行する。
68. `ranking` に `pd.DataFrame(rows)` の結果を代入する。
69. `ranking` に `ranking.sort_values(['feasible', 'elapsed_hours'], ascending=[False, True], na_position='last', kind='stable')` の結果を代入する。
70. `output.mkdir(...)` を実行する。
71. `ranking.to_csv(...)` を実行する。
72. `feasible` に `ranking.loc[ranking['feasible'].fillna(False)]` の結果を代入する。
73. `selection` に `{'scope': 'fixed-policy exact 1 Hz simulation ranking; mesh convergence and model validation remain separate gates', 'source_stage': str(args.stage), 'candidate_count': int(len(ranking)), 'feasible_candidate_count': int(len(feasible)), 'selected': False}` の結果を代入する。
74. 条件 `not feasible.empty` を判定し、真なら内部処理を行う。
75. `winner` に `feasible.iloc[0]` の結果を代入する。
76. `selected_policy` に `output / 'selected_exact_policy.csv'` の結果を代入する。
77. `shutil.copy2(...)` を実行する。
78. `selection.update(...)` を実行する。
79. `(output / 'exact_selection.json').write_text(...)` を実行する。
80. `print(...)` を実行する。

代表コード断片

```
def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--profile", type=Path, required=True)
    parser.add_argument("--campaign-dir", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--stage", default="control_1km")
    parser.add_argument(
        "--seed-label",
        default="",
        help="Evaluate only one seed directory (for parallel acceptance workers).",
    )
    parser.add_argument("--integration-ds-km", type=float, default=0.1)
    parser.add_argument("--control-ds-km", type=float, default=1.0)
    parser.add_argument("--no-resume", action="store_true")
    args = parser.parse_args()
```

```

base_profile = args.profile.resolve()
campaign = args.campaign_dir.resolve()
output = args.output_dir.resolve()
seed_pattern = args.seed_label.strip() or "seed_*"
policies = sorted(campaign.glob(f"{seed_pattern}/{args.stage}/latest_policy.csv"))
if not policies:
    raise FileNotFoundError(f"No {args.stage} policies under {campaign}")
base_cfg = yaml.safe_load(base_profile.read_text(encoding="utf-8")) or {}
model = base_cfg["model"]
rows = []

for policy in policies:
    label = policy.parents[1].name
    run_dir = output / label
    profile_path = run_dir / "profile_exact_1hz.yaml"
    manifest_path = run_dir / "latest_simulation_run.json"
    signature_path = run_dir / "run_signature.txt"
    cfg = materialize_profile(
        base_profile,
    ...

```

7.9 CLI 引数

行	引数
251	--profile
252	--campaign-dir
253	--output-dir
254	--stage
255	--seed-label
260	--integration-ds-km
261	--control-ds-km
262	--no-resume

8 処理の流れ

1. CLI 引数を解釈し、main() から処理を起動する。

9 このファイルの位置づけ

この資料群では、scripts/validate_gpu_upper_policy_candidates.py を **planning validation** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の profile / launch / telemetry と後段の planner / logger / report へ繋がる。